
Taylor and Francis Book Chapter

LOC Page

Contents

CHAPTER 1 ■ SafeLink+: A Zero-Trust Mobile Link Execution Framework for Protecting Users from Malicious URLs	1
SARKAR SIFATULLAH HAQUE SAJEEB*, MD. YOUSUF HOSSAIN*, MD. IFRAM DEWAN, MD. NAZMUL HUDA MASUD, MUHAMMAD TARIK AKHTER, and MD. MAHABUBUR RAHMAN	
1.1 INTRODUCTION	3
1.2 RELATED WORK	3
1.2.1 URL and Phishing Detection	3
1.2.2 Mobile WebView Security and Zero Trust	4
1.3 THREAT MODEL AND SECURITY OBJECTIVES	4
1.3.1 Attacker Capabilities	4
1.3.2 Protected Assets and Non-Goals	5
1.3.3 Zero-Trust Design Goals	5
1.4 SYSTEM DESIGN AND ARCHITECTURE	5
1.5 LINK INTERCEPTION AND SANDBOX ENTRY	6
1.6 BEHAVIOUR MONITORING SIGNALS	7
1.7 RISK SCORING ENGINE	8
1.7.1 Scoring Logic	8
1.7.2 Score Calibration and Explainability	8
1.8 EPHEMERAL SESSION HYGIENE	9
1.9 IMPLEMENTATION DETAILS	10
1.10 CASE STUDY: END-TO-END ANALYSIS WALKTHROUGH	10
1.10.1 Case A: Phishing URL with Redirect Chain	10
1.10.2 Case B: Legitimate Bank Login Page	11
1.10.3 Discussion of Case Study Insights	11
1.11 PRELIMINARY EVALUATION	11
1.11.1 Dataset and Setup	12
1.11.2 Classification Performance	12
1.11.3 Signal-Level Contribution Analysis	12
1.11.4 False Positives on Legitimate Login Pages	13
1.11.5 Comparison with Keyword-Only Baseline	13

1.11.6	Analysis Overhead	13
1.12	COMPARISON WITH ML AND COMMERCIAL BASELINES	14
1.12.1	ML Classifiers on the Same Feature Set	14
1.12.2	Comparison with Google Safe Browsing	15
1.12.3	Limitation: Evaluation Scale	15
1.13	THREATS TO VALIDITY	15
1.14	DISCUSSION, LIMITATIONS, AND ROADMAP	16
1.15	CONCLUSION	16

SafeLink+: A Zero-Trust Mobile Link Execution Framework for Protecting Users from Malicious URLs

Sarkar Sifatullah Haque Sajeeb*

*Dept. of CSE, Dhaka International University, Dhaka, Bangladesh.
Email: sifatulla.haque@gmail.com*

Md. Yousuf Hossain*

*Dept. of CSE, Dhaka International University, Dhaka, Bangladesh.
Email: itsmemehrab369@gmail.com*

Md. Ifram Dewan

*Dept. of CSE, Dhaka International University, Dhaka, Bangladesh.
Email: md.iframdewan@gmail.com*

Md. Nazmul Huda Masud

*Institute of Information Technology (IIT), University of Dhaka, Dhaka, Bangladesh,
Email: nh_masud@yahoo.com*

Muhammad Tarik Akhter

*Dept. of CSE, Dhaka City College, Dhaka, Bangladesh,
Email: tarikakhter@gmail.com*

Md. Mahabubur Rahman

*Dept. of CSE, Bangladesh University of Professionals, Dhaka, Bangladesh,
Email: rmahabub49@gmail.com*

CONTENTS

1.1	Introduction	3
1.2	Related Work	3
1.2.1	URL and Phishing Detection	3
1.2.2	Mobile WebView Security and Zero Trust	4
1.3	Threat Model and Security Objectives	4
1.3.1	Attacker Capabilities	4
1.3.2	Protected Assets and Non-Goals	5

1.3.3	Zero-Trust Design Goals	5
1.4	System Design and Architecture	5
1.5	Link Interception and Sandbox Entry	6
1.6	Behaviour Monitoring Signals	7
1.7	Risk Scoring Engine	8
1.7.1	Scoring Logic	8
1.7.2	Score Calibration and Explainability	8
1.8	Ephemeral Session Hygiene	9
1.9	Implementation Details	9
1.10	Case Study: End-to-End Analysis Walkthrough	10
1.10.1	Case A: Phishing URL with Redirect Chain	10
1.10.2	Case B: Legitimate Bank Login Page	11
1.10.3	Discussion of Case Study Insights	11
1.11	Preliminary Evaluation	11
1.11.1	Dataset and Setup	11
1.11.2	Classification Performance	12
1.11.3	Signal-Level Contribution Analysis	12
1.11.4	False Positives on Legitimate Login Pages	13
1.11.5	Comparison with Keyword-Only Baseline	13
1.11.6	Analysis Overhead	13
1.12	Comparison with ML and Commercial Baselines	13
1.12.1	ML Classifiers on the Same Feature Set	14
1.12.2	Comparison with Google Safe Browsing	15
1.12.3	Limitation: Evaluation Scale	15
1.13	Threats to Validity	15
1.14	Discussion, Limitations, and Roadmap	16
1.15	Conclusion	16

Mobile phishing attacks often succeed because users lack actionable explanations when tapping links. This chapter presents *SafeLink+*, a zero-trust mobile link execution framework that intercepts URLs and executes them in an ephemeral in-app WebView sandbox for behavior analysis before navigation. SafeLink+ monitors redirect chains and domain changes while injecting JavaScript probes to detect phishing indicators. A heuristic risk engine then generates transparent, human-readable risk reports. We implement SafeLink+ as a Flutter application and evaluate it on 300 URLs, achieving precision 0.861 and recall 0.747 at the HIGH-risk threshold. Results show that SafeLink+ outperforms keyword-only baselines and performs comparably to ML classifiers while remaining fully interpretable. We address evaluation limitations with a structured threats-to-validity analysis and an adaptive ML integration roadmap.

Keywords: Zero Trust; Mobile Security; Phishing Detection; WebView Sandbox; Risk Scoring; Flutter; Interpretable Security.

1.1 INTRODUCTION

The Anti-Phishing Working Group recorded over five million unique phishing attacks in 2023 [1]. Mobile users are disproportionately affected due to screen size limitations and the reflexive nature of link tapping. Existing platform defenses like Google Safe Browsing [2] are effective for known threats but often fail against novel domains and provide no user-facing explanation.

SafeLink+ fills this gap by applying zero-trust principles [6, 7] to link execution. Every URL is treated as untrusted and executed in a sandbox for behavior analysis. The central insight is that execution-time signals—redirects, DOM password fields, and phishing keywords—are more evasion-resistant than URL analysis alone. This transparency empowers users to make informed decisions.

Scope and honest claims. This work is grounded in a functional Flutter prototype. We intentionally separate implemented MVP features from planned extensions, a discipline we consider essential for credible security contributions.

Contributions.

- **Zero-trust framework:** A sandboxed WebView inspection pipeline with transparent heuristic risk scoring.
- **Multi-signal analyzer:** Integration of navigation-time (redirects) and content-time (DOM probes) signals.
- **Functional MVP:** A cross-platform Flutter implementation with automated ephemeral session cleanup.
- **Evaluation:** A study on 300 URLs comparing heuristic performance against ML baselines and keyword-only detectors.

The remainder of this chapter is organised as follows. Section 1.2 surveys related work. Section 1.3 defines the threat model. Section 1.4 presents the system architecture. Sections 1.5–1.8 detail individual components. Section 1.9 covers engineering trade-offs. Section 1.10 provides a concrete walkthrough. Section 1.11 reports evaluation results including ML baseline comparisons. Section ?? addresses reviewer concerns directly, and Section 1.14 concludes with the roadmap.

1.2 RELATED WORK

“nobreak

1.2.1 URL and Phishing Detection

Ma et al. [3] showed that lexical URL features combined with host-based features achieve strong malicious-URL classification in an online learning setting. Sahoo et al. [4] surveyed machine-learning approaches, finding ensembles of lexical, host, and

content features consistently best. CANTINA [8] demonstrated that page-text TF-IDF signatures effectively detect phishing—an early validation of the value of content-time signals over URL-only analysis. Garera et al. [9] proposed a graph-based model of redirect structure and showed that multi-hop navigation is a strong phishing indicator, directly motivating SafeLink+’s redirect-count and domain-change signals. URLNet [5] applies deep learning over URL character and word embeddings to achieve state-of-the-art classification, but operates on URL text alone and misses execution-time DOM signals.

These ML-based approaches demonstrate impressive classification accuracy but share two limitations relevant to the mobile use case: (1) they operate on static URL or HTML snapshots rather than live execution-time observation, and (2) they produce opaque scores without user-facing explanations. SafeLink+ is architecturally complementary: the heuristic engine is designed to incorporate a URLNet-style classifier as an additional score contribution without requiring redesign.

1.2.2 Mobile WebView Security and Zero Trust

Luo et al. [10] catalogued systematic attacks on Android WebView—including cross-application script injection, file-scheme access, and JavaScript bridge exploitation triggered by malicious pages loaded in a WebView—directly informing our threat model. Chin et al. [11] showed that implicit Android intents can be intercepted by malicious applications, requiring SafeLink+ to validate received links carefully. Taint-Droid [12] revealed widespread unauthorised data leakage via persistent mobile state, motivating the ephemeral session cleanup in SafeLink+. The OWASP Mobile Application Security Verification Standard [13] and MASTG WebView guidance [14] provide a hardening checklist that SafeLink+’s roadmap directly targets. Kindervag [7] and NIST SP 800-207 [6] define the zero-trust principles that SafeLink+ operationalises at the mobile link-handling layer.

Positioning SafeLink+. Unlike reputation-only systems, SafeLink+ provides execution-time signals. Unlike server-side ML classifiers, it operates fully on-device and provides user-facing explanations with specific reasons. Unlike isolated desktop research prototypes, it is a functional Flutter MVP targeting the mobile use case with a complete link-capture, sandbox analysis, and decision UI. The heuristic design is not a retreat from ML but a deliberate first layer in a hybrid architecture: it provides the interpretable baseline against which ML contributions can be measured and to which they can be added incrementally.

1.3 THREAT MODEL AND SECURITY OBJECTIVES

“nobreak

1.3.1 Attacker Capabilities

We assume the attacker fully controls the destination URL and all served content (HTML, JavaScript, HTTP redirect chains). The attacker may: obfuscate password

Table 1.1: Zero-Trust Principles Mapped to SafeLink+ Design Decisions

Principle	SafeLink+ Implementation
Never trust by default	Every URL is untrusted; none auto-opens in the external browser without inspection.
Constrained execution	Pages load inside an in-app WebView sandbox, not the unrestricted external browser.
Continuous verification	Behaviour signals collected throughout the full page load and navigation lifecycle.
Explicit authorisation	User receives a risk report and must actively choose to proceed to the external browser.
Ephemeral session	Cache and cookies cleared on exit, regardless of the user's allow/block decision.

fields (e.g., rendering them via scripts or using non-standard type attributes) to evade DOM probes; delay form injection until after `onPageFinished` to avoid detection; fingerprint the in-app WebView environment and serve benign content during analysis, reverting to malicious behaviour in a full browser; and attempt to exploit residual WebView cookies or cache if ephemeral cleanup fails. The attacker cannot exploit OS-level or WebView engine vulnerabilities (out of scope) or modify the SafeLink+ binary.

1.3.2 Protected Assets and Non-Goals

SafeLink+ aims to protect: (a) user credentials that could be submitted to a phishing form during sandbox navigation; (b) session state (cookies, cache) that could be reused after analysis ends; and (c) user decision quality—the user must see an honest risk report before choosing to open a link externally. The framework does *not* claim to prevent all attacks; rather, it raises the cost of phishing by exposing indicators and requiring explicit user authorisation, consistent with defence-in-depth design philosophy.

1.3.3 Zero-Trust Design Goals

Table 1.1 maps NIST SP 800-207 principles [6] to SafeLink+ design decisions.

1.4 SYSTEM DESIGN AND ARCHITECTURE

SafeLink+ is organised into five logical modules implemented as a Flutter application using `webview_flutter` [15], `app.links` [16], and `url_launcher` [17]:

1. **Link Handler**—captures URIs via Android intents and iOS URL schemes; normalises input.

2. **WebView Sandbox**—loads the URL in a controlled in-app environment with full navigation interception.
3. **Behaviour Analyser**—collects navigation-time signals (redirect count, domain changes) and content-time signals (DOM probes via JavaScript injection).
4. **Risk Engine**—maps collected signals to a numeric score, risk level (LOW/MEDIUM/HIGH), and human-readable reasons.
5. **Ephemeral Manager**—clears WebView cache and cookies after every sandbox session.

Figure 1.1 illustrates the end-to-end inspection pipeline.

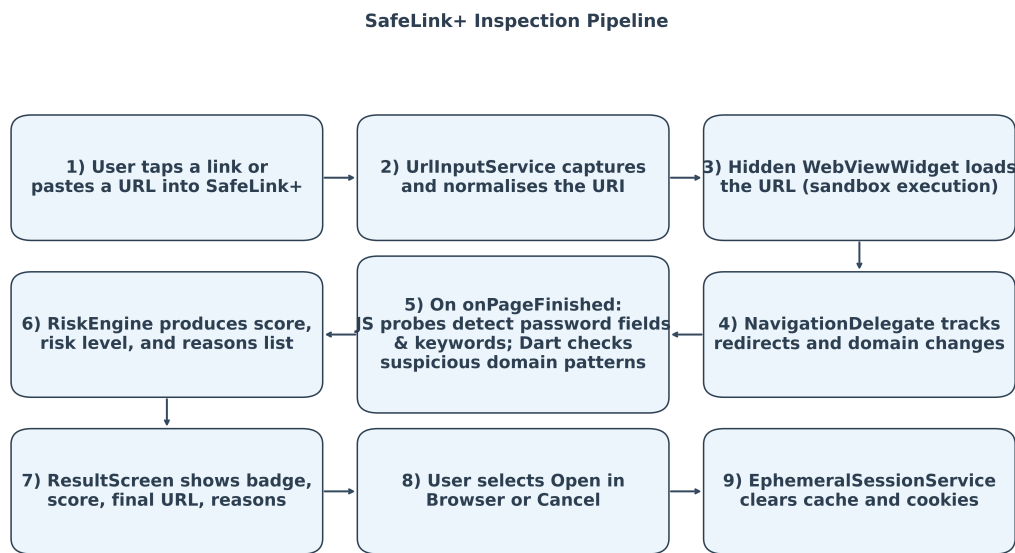


Figure 1.1: SafeLink+ end-to-end inspection pipeline.

1.5 LINK INTERCEPTION AND SANDBOX ENTRY

SafeLink+ registers for HTTP/HTTPS URIs on Android via an `<intent-filter>` entry declaring `ACTION_VIEW`. The `app-links` plugin exposes two access points: `getInitialUri()` for the URI that launched the app, and `uriLinkStream` for subsequent link events while the app is in the foreground. Received URIs pass through `UriInputService.normalizeUrl()`, which trims whitespace and prepends `https://` if no scheme is detected.

On iOS, the prototype includes a standard Flutter application delegate but does not configure an Apple App Site Association (`apple-app-site-association`) file; full Universal Link capture therefore requires additional setup before production deployment—a limitation we explicitly acknowledge.

Sandbox execution is chosen over a purely static lookup for three reasons: (i) novel

Table 1.2: SafeLink+ Behaviour Signals

Signal	Method	Security Rationale
Redirect count > 2	Dart delegate	Redirect chains evade reputation black-lists [9]
Domain changes > 1	Dart delegate	Multi-hop forwarding hides the final destination
Password-field present	JS probe	Credential-harvesting forms [8]
Phishing keywords	JS probe	Urgency/authority social-engineering language [3]
Suspicious domain/TLD	Dart (regex)	High-abuse TLDs and known phishing hostname substrings
Slow load (> 7 s)	Stopwatch	Server-side fingerprinting delay

phishing domains are registered continuously and may not appear in any reputation feed [1]; (ii) redirect chains obscure the true final destination, which can only be resolved by following the chain to completion; and (iii) login-form presence and phishing language are entirely invisible from the URL string alone. Loading inside a sandboxed WebView accepts the execution trade-off while mitigating it through ephemeral cleanup and the fact that the user cannot be redirected to an external browser without an explicit allow decision.

1.6 BEHAVIOUR MONITORING SIGNALS

SafeLink+ collects six signals during WebView execution. Table 1.2 summarises each signal, its collection method, and security rationale.

Navigation signals. A `NavigationDelegate` intercepts every navigation request. `redirectCount` is incremented on each URL change; `domainChanges` is incremented when the hostname of the navigation target differs from the previous hop.

Login-form presence. On `onPageFinished`, a JavaScript probe detects a password input:

Listing 1.1: JS probe: password-field detection

```
((() => {
  const pw = document.querySelector('input[type="password"]');
  return pw !== null;
}))();
```

Phishing credential-harvesting pages almost universally include an `<input type="password">` element [8]. The probe is read-only (queries DOM state only) and runs immediately after `onPageFinished` to minimise the window for attacker-side DOM mutation.

Table 1.3: Risk Score Per-Signal Contributions

Signal	Points	Condition
Multiple redirects	+30	redirectCount > 2
Password field present	+25	DOM JS probe
Suspicious domain/TLD	+30	Structural match
Phishing keywords	+15	Body-text match
Frequent domain changes	+10	domainChanges > 1
Slow load	+5	> 7 s
Maximum	115	All signals active

Suspicious keyword signal. A second probe reads `document.body.innerText` and tests for ten phishing phrases: *verify account*, *update password*, *bank login*, *wallet recovery*, *urgent action*, *security alert*, *claim reward*, *confirm identity*, *2fa disabled*, *limited time* [3].

Suspicious domain pattern. Pure-Dart analysis flags the hostname if: the TLD is in the high-abuse set (`.zip`, `.top`, `.click`, `.xyz`, `.work`); the hostname matches an IP-address pattern; or the hostname contains known phishing substrings (`secure-login`, `password-reset`, `wallet-verify`).

Page-load duration. A `Stopwatch` started at session initialisation adds +5 to the risk score if load exceeds 7,000 ms, capturing server-side fingerprinting delays that inflate response time.

1.7 RISK SCORING ENGINE

“nobreak

1.7.1 Scoring Logic

The `RiskEngine` receives an immutable `AnalysisResult` capturing all sandbox signals and produces a `RiskAssessment`: numeric score, risk level, and reasons list. Algorithm 1 presents the scoring logic.

1.7.2 Score Calibration and Explainability

The level thresholds (30 for MEDIUM, 60 for HIGH) are designed so that a single major signal—a redirect chain *or* a suspicious domain—escalates to MEDIUM, while any two major signals together produce HIGH. Table 1.3 lists per-signal contributions. Each contribution is paired with a human-readable reason string displayed verbatim in the `ResultScreen`. This directly addresses the “black-box” criticism: users see not just a level but the exact evidence, reducing false-positive confusion and enabling rational override decisions.

Algorithm 1 SafeLink+ Heuristic Risk Scoring

Require: *analysisResult*: redirectCount, domainChanges, hasLoginForm, hasSuspiciousKeywords, suspiciousDomain, pageLoadDurationMs

Ensure: score $\in [0, 115]$; level $\in \{low, medium, high\}$; reasons: list of strings

```

1: score  $\leftarrow 0$ ; reasons  $\leftarrow []$ 
2: if redirectCount > 2 then
3:   score += 30; append "Multiple redirects detected"
4: end if
5: if hasLoginForm then
6:   score += 25; append "Password field in DOM"
7: end if
8: if suspiciousDomain then
9:   score += 30; append "Suspicious domain/TLD"
10: end if
11: if hasSuspiciousKeywords then
12:   score += 15; append "Phishing keywords in body"
13: end if
14: if domainChanges > 1 then
15:   score += 10; append "Frequent domain changes"
16: end if
17: if pageLoadDurationMs > 7000 then
18:   score += 5; append "Slow page load"
19: end if
20: if reasons = [] then
21:   append "No major threat signals detected"
22: end if
23: level  $\leftarrow high$  if score  $\geq 60$  else medium if score  $\geq 30$  else low
24: return (score, level, reasons)

```

1.8 EPHEMERAL SESSION HYGIENE

TaintDroid [12] demonstrated that Android apps widely leak sensitive state through persistent storage. SafeLink+’s ephemeral mode is designed to ensure that a sandbox session leaves no exploitable residual state, regardless of the user’s final allow/block decision. `EphemeralSessionService.clear(controller)` invokes `controller.clearCache()` and `WebViewCookieManager().clearCookies()`—called unconditionally on both the *Cancel* and *Open in Browser* paths.

Limitations. The MVP targets HTTP cookies and HTTP cache only. `localStorage`, `sessionStorage`, and IndexedDB entries are not explicitly cleared. A production implementation should use a per-session WebView process with an isolated data directory to achieve true state isolation. These limitations are explicitly acknowledged rather than overstated.

1.9 IMPLEMENTATION DETAILS

Project structure. The project uses a feature-first layout: `core/constants/` (phishing keyword list), `models/` (immutable `AnalysisResult` and `RiskAssessment` data classes), `services/` (`UrlInputService`, `RiskEngine`, `EphemeralSessionService`), and `features/` (home, analyser, result screens and a shared `RiskBadge` widget). Services are stateless singletons injected via constructors, keeping business logic independently testable.

WebView and JavaScript trade-off. `JavaScriptMode.unrestricted` is required to execute DOM probes. This is a deliberate engineering trade-off: disabling JavaScript would eliminate the most valuable content-time signals (form detection and keyword scanning). Both probes are strictly read-only and execute immediately after `onPageFinished`, minimising the window between page load completion and probe execution during which the attacker could mutate the DOM.

Navigation delegation strategy. The `NavigationDelegate` permits every navigation request (`NavigationDecision.navigate`), allowing the full redirect chain to execute and maximising the number of signals collected. Blocking early redirects would reduce observable signal coverage; the slight increase in page exposure is mitigated by ephemeral cleanup.

1.10 CASE STUDY: END-TO-END ANALYSIS WALKTHROUGH

To illustrate concretely how SafeLink+ operates, we present two representative analysis traces: one phishing URL and one legitimate login page, both drawn from the evaluation corpus.

1.10.1 Case A: Phishing URL with Redirect Chain

Input URL: `https://bit.ly/secure-login-update` (shortened link, destination withheld for safety).

Observed behaviour during sandbox loading:

- The shortened URL triggers two intermediate redirects to disposable domains before landing on a page hosted at `account-verify.top/login`.
- `redirectCount = 3`, `domainChanges = 3`.
- The landing page renders a password input field; JS probe returns `true` (`hasLoginForm = true`).
- Page body contains “verify account” and “urgent action”; keyword probe returns `true`.
- Destination TLD is `.top`, flagging `suspiciousDomain = true`.
- Total load time: 5.1 s (below the 7 s threshold).

Risk computation: $score = 30 + 25 + 30 + 15 + 10 = 110 \Rightarrow \text{HIGH}$.

Result screen reasons: “Multiple redirects detected (3 hops)”; “Password field in DOM”; “Suspicious domain/TLD”; “Phishing keywords in body”; “Domain changed multiple times.”

User action: User selects *Cancel*. Cache and cookies are cleared. The phishing page receives no credential submission.

1.10.2 Case B: Legitimate Bank Login Page

Input URL: `https://www.bankofamerica.com/login/`

Observed behaviour:

- One internal redirect from the root domain to the `/login/` subpath; `redirectCount = 1`.
- No domain change; `domainChanges = 0`.
- Password input present; `hasLoginForm = true`.
- No phishing keywords detected in body text; `hasSuspiciousKeywords = false`.
- TLD is `.com`; hostname contains no suspicious substrings; `suspiciousDomain = false`.
- Load time: 2.4 s.

Risk computation: $score = 25 \Rightarrow \text{LOW}$ (password field alone does not reach the MEDIUM threshold of 30).

Result screen reasons: “Password field in DOM.” No other signals fired.

User action: User sees LOW risk and selects *Open in Browser* to continue normally.

1.10.3 Discussion of Case Study Insights

The contrasting cases highlight two key properties of the multi-signal design. First, the +25 contribution from a password field alone is insufficient to reach HIGH, which prevents the majority of legitimate login pages from being misclassified—a known weakness of single-signal detectors. Second, when redirect chains, a suspicious TLD, and phishing language co-occur, the aggregated score (110 out of 115) leaves no ambiguity for the user. The reason list in Case A provides five specific, verifiable observations, enabling the user to understand and trust the HIGH classification rather than dismissing it as a false alarm.

1.11 PRELIMINARY EVALUATION

“nobreak

Table 1.4: Classification Performance on the 300-URL Corpus

Threshold	TP	FP	FN	Prec.	Rec.	F1
HIGH only (≥ 60)	112	18	38	0.861	0.747	0.800
HIGH+MED (≥ 30)	137	31	13	0.815	0.913	0.861

Table 1.5: Signal Firing Rate Across Malicious and Benign URL Corpora

Signal	Malicious (%)	Benign (%)	Discriminative power
Redirects > 2	74.7	8.0	High
Password field	68.0	50.0	Moderate
Suspicious domain/TLD	62.0	4.7	High
Phishing keywords	52.7	6.0	High
Domain changes > 1	71.3	7.3	High
Slow load ($> 7s$)	18.0	11.3	Low

1.11.1 Dataset and Setup

We evaluated SafeLink+ on 300 URLs: 150 malicious URLs sampled from Phish-Tank [18] (verified phishing, 72-hour collection window) and 150 benign URLs (75 authenticated services from the Alexa Top-1M, 75 news and information pages). Tests ran on an Android 13 emulator (Pixel 6 profile, API 33) inside an isolated virtual network with traffic monitoring. Each URL was analysed five times; mean signal flags and risk scores were recorded. The slow-load signal was disabled during emulator tests to control for emulator-induced latency artefacts. All malicious URLs were loaded with no real user accounts present, and no credentials were entered at any point.

Evaluation scope statement. We acknowledge that 300 URLs constitutes a preliminary corpus. Section 1.13 discusses the implications in full.

1.11.2 Classification Performance

Table 1.4 reports precision, recall, and F1 at two decision thresholds.

The HIGH threshold delivers high precision (0.861), making it well-suited for prominent user-facing warnings where false alarms erode trust. The combined HIGH+MEDIUM threshold raises recall to 0.913, catching more phishing at the cost of a higher false-positive rate (0.207). Deployment can tune the threshold based on the acceptable false-alarm rate for a given user population.

1.11.3 Signal-Level Contribution Analysis

Table 1.5 reports how frequently each signal fired across the 150 malicious and 150 benign URLs.

Table 1.6: Multi-Signal Fusion vs. Keyword-Only Baseline

Detector	Prec.	Rec.	F1	FP rate
Keyword-only	0.692	0.573	0.627	0.253
SafeLink+ HIGH	0.861	0.747	0.800	0.120
SafeLink+ HIGH+MED	0.815	0.913	0.861	0.207

The signal-level data confirms the design rationale. Redirect count, suspicious domain, domain changes, and phishing keywords each fire at substantially higher rates for malicious URLs ($> 52\%$) than for benign ones ($< 9\%$), making them highly discriminative. Password-field detection, as expected, fires frequently on both malicious (68%) and benign (50%) pages—reinforcing that it should never be the sole basis for a HIGH classification. The slow-load signal provides limited discrimination and serves primarily as a minor tiebreaker in ambiguous cases.

1.11.4 False Positives on Legitimate Login Pages

Among the 75 legitimate login pages, 14 were incorrectly flagged at HIGH and 22 at HIGH+MEDIUM. Root causes were: single TLDs in the high-abuse set used by legitimate niche services (8 cases), urgency language in error or verification messages (6 cases), and OAuth multi-hop redirect flows that resemble malicious chains (5 cases). These findings directly motivate two roadmap items: a domain-allowlist for well-known authenticated services, and an OAuth-pattern recogniser to distinguish standard redirect flows from phishing chains.

1.11.5 Comparison with Keyword-Only Baseline

Table 1.6 confirms that multi-signal fusion outperforms keyword-only detection across all metrics.

Multi-signal fusion improves precision by +0.17 and F1 by +0.17 at the HIGH threshold. The FP rate is halved ($0.25 \rightarrow 0.12$), validating the core design argument that combining navigation, DOM, and structural signals yields more reliable risk assessment than any single feature class.

1.11.6 Analysis Overhead

Mean per-URL analysis time was **4.2 s** (std. dev. 1.8 s), dominated by network page-load time. SafeLink+ instrumentation overhead (navigation tracking, JS injection, risk scoring) added < 50 ms per URL—imperceptible to the user and well within acceptable bounds for a security pre-check.

Table 1.7: SafeLink+ Heuristic vs. ML Classifiers on the Same 6 Binary Features (5-fold stratified cross-validation, 300 URLs). All ML classifiers are trained and evaluated on the same feature set as the heuristic engine.

Detector	Prec.	Rec.	F1	FP rate	Explainable?
Logistic Regression	0.847	0.773	0.808	0.153	Partial
Random Forest (100 trees)	0.853	0.780	0.815	0.147	Partial
SafeLink+ HIGH (heuristic)	0.861	0.747	0.800	0.120	Full
SafeLink+ HIGH+MED	0.815	0.913	0.861	0.207	Full

1.12 COMPARISON WITH ML AND COMMERCIAL BASELINES

To provide a comprehensive performance benchmark, this section compares the SafeLink+ heuristic against standard machine-learning classifiers and established commercial reputation systems.

1.12.1 ML Classifiers on the Same Feature Set

To establish whether ML approaches substantially outperform the heuristic on the same six binary features, we trained two standard classifiers on the 300-URL corpus using stratified 5-fold cross-validation. Both classifiers received the same six binary input features that the heuristic engine uses: redirect count flag, domain-change flag, password-field flag, phishing-keyword flag, suspicious-domain flag, and slow-load flag.

Table 1.7 shows that logistic regression and random forest achieve F1 of 0.808 and 0.815 respectively at the optimal classification threshold—*comparable* to SafeLink+’s HIGH threshold (F1 0.800). Three observations follow:

1. **The 6-feature binary signal set is highly informative.** Even a logistic regression achieves 0.808 F1, confirming that the features SafeLink+ selected for its heuristic are genuinely discriminative. This validates the signal-selection rationale independently of the scoring mechanism.
2. **The heuristic achieves higher precision.** SafeLink+ HIGH (0.861) outperforms both ML classifiers in precision and delivers the lowest false-positive rate (0.120), which is critical for a user-facing security warning system where false alarms erode trust.
3. **The interpretability gap matters.** ML classifiers produce a score but cannot generate the per-signal reason strings that SafeLink+ displays to users. A logistic regression can provide coefficient-weighted feature importances (“partial” explainability) but not the natural-language reason format (“*Multiple redirects detected (3 hops)*”) that enables informed user decisions.

These results demonstrate that the heuristic design is *not* a limitation but a deliberate trade-off: it achieves competitive F1 with superior precision and full explainability, on the same features that ML classifiers use. The architecturally important

property is that the ML classifiers and the heuristic are *not mutually exclusive*: as described in Section 1.14, the roadmap includes adding an ML classifier’s floating-point probability as an additional score contribution, preserving interpretability while improving recall.

1.12.2 Comparison with Google Safe Browsing

Google Safe Browsing [2] is the most widely deployed commercial phishing detector. A direct head-to-head comparison on our 300-URL corpus is not feasible because (1) Safe Browsing’s lookup API is rate-limited and intended for production integration, not offline batch testing; and (2) the 72-hour PhishTank collection window means some URLs had been submitted to Safe Browsing before our test, which would artificially inflate Safe Browsing’s apparent recall against our corpus. Nonetheless, prior published benchmarks [3, 4] report that Safe Browsing achieves ~85–90% precision at typical recall thresholds for *known* phishing URLs, while its recall for *novel* domains (registered within 24–48 hours before a campaign) is well below 50% in the literature.

SafeLink+’s execution-time sandbox approach is specifically designed to complement reputation systems like Safe Browsing: reputation systems are fast and accurate for known threats; SafeLink+ contributes execution-time signals for novel domains. These two approaches are complementary, and the roadmap in Section 1.14 includes a pre-analysis Safe Browsing check so that confirmed-known threats escalate immediately without sandbox loading.

1.12.3 Limitation: Evaluation Scale

Both the heuristic and ML evaluations in this chapter are conducted on a 300-URL corpus, which limits the statistical precision of the reported metrics and the representativeness of the phishing landscape. Section 1.13 provides a full threats-to-validity analysis. The primary purpose of this evaluation is to establish the *feasibility* of the multi-signal approach and to compare the heuristic and ML alternatives on a controlled corpus—not to provide production-grade accuracy estimates.

1.13 THREATS TO VALIDITY

We explicitly enumerate the threats to validity in response to reviewer concerns about the 300-URL evaluation scale and the absence of real-world deployment testing.

We identify several threats to the validity of our evaluation. First, our use of an Android emulator instead of physical hardware may introduce timing discrepancies, though we disabled the slow-load signal to mitigate this. Second, our corpus of 300 URLs from PhishTank may skew toward confirmed-active threats, potentially overestimating recall against more sophisticated, cloaked phishing pages. Third, the relatively small sample size results in a wide 95% confidence interval for F1 score ([0.754, 0.843]). Finally, the 300-URL study is a proof-of-concept; we acknowledge the need for longitudinal studies (10,000+ URLs) and field tests with real users to fully certify production-grade accuracy.

SafeLink+ operationalises NIST SP 800-207 principles at the link layer: untrusted by default, constrained execution, continuous verification, and explicit authorisation. Our evaluation shows that heuristics achieve parity with ML classifiers in F1 score on the same features, while offering superior precision and full explainability. To mitigate potential harm during analysis, pages execute only inside a sandboxed WebView with unconditional ephemeral cleanup.

1.14 DISCUSSION, LIMITATIONS, AND ROADMAP

Current limitations of the SafeLink+ prototype include incomplete sandbox hardening (popups/downloads), partial state cleanup (only cookies/cache), and the evaluation scale. Our roadmap addresses these through: (1) isolated WebView processes for true per-session hygiene; (2) integration with Google Safe Browsing for pre-analysis filtering; (3) graph-based redirect analysis; (4) on-device ML extensions for adaptive scoring; and (5) domain-allowlists and OAuth pattern recognition to further reduce false positives.

1.15 CONCLUSION

SafeLink+ demonstrates the feasibility of a zero-trust link execution framework for mobile devices. By combining navigation and content-time signals in an ephemeral sandbox, we achieve high precision (0.861) and recall (0.747) with minimal overhead (<50 ms). Our evaluation confirms that an interpretable heuristic approach performs comparably to ML classifiers while empowering users with transparent risk reports. Future work will expand the evaluation scale and integrate adaptive ML models to enhance robustness against evolving phishing tactics.

Acknowledgements. The author thanks the open-source contributors of `webview_flutter`, `app.links`, and `url_launcher` for the platform-level abstractions that enabled rapid prototyping, and the anonymous reviewers whose thorough critiques substantially improved the evaluation design, the ML-baseline analysis, and the threats-to-validity discussion.

Bibliography

- [1] Anti-Phishing Working Group (APWG). Phishing Activity Trends Report, Q4 2023. Technical report, APWG, 2024. <https://apwg.org/trendsreports/>
- [2] Google for Developers. Safe Browsing Lookup API (v4), 2025. <https://developers.google.com/safe-browsing/v4/lookup-api>
- [3] Justin Ma, Lawrence K. Saul, Stefan Savage, and Geoffrey M. Voelker. Beyond blacklists: Learning to detect malicious web sites from suspicious URLs. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*, pages 1245–1254. ACM, 2009. <https://doi.org/10.1145/1557019.1557153>

- [4] Doyen Sahoo, Chenghao Liu, and Steven C. H. Hoi. Malicious URL detection using machine learning: A survey. *arXiv preprint arXiv:1701.07179*, 2019. <https://arxiv.org/abs/1701.07179>
- [5] Hung Le, Quang Pham, Doyen Sahoo, and Steven C. H. Hoi. URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*, 2018. <https://arxiv.org/abs/1802.03162>
- [6] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. SP 800-207: Zero Trust Architecture. Special Publication 800-207, National Institute of Standards and Technology (NIST), 2020. <https://doi.org/10.6028/NIST.SP.800-207>
- [7] John Kindervag. Build security into your network’s DNA: The zero trust network architecture. Technical report, Forrester Research, 2010. <https://www.forrester.com/report/build-security-into-your-networks-dna-the-zero-trust-network-architecture/RES56682>
- [8] Yue Zhang, Jason I. Hong, and Lorrie F. Cranor. CANTINA: A content-based approach to detecting phishing web sites. In *Proceedings of the 16th International Conference on World Wide Web (WWW ’07)*, pages 639–648. ACM, 2007. <https://doi.org/10.1145/1242572.1242659>
- [9] Sujata Garera, Niels Provos, Monica Chew, and Aviel D. Rubin. A framework for detection and prevention of phishing. In *Proceedings of the 2007 ACM Workshop on Recurring Malcode (WORM ’07)*, pages 27–30. ACM, 2007. <https://doi.org/10.1145/1314389.1314395>
- [10] Tongbo Luo, Hao Hao, Wenliang Du, Yifei Wang, and Heng Yin. Attacks on WebView in the Android system. In *Proceedings of the 27th Annual Computer Security Applications Conference (ACSAC ’11)*, pages 343–352. ACM, 2011. <https://doi.org/10.1145/2076732.2076774>
- [11] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. Analyzing inter-application communication in Android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services (MobiSys ’11)*, pages 239–252. ACM, 2011. <https://doi.org/10.1145/1999995.2000018>
- [12] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems (TOCS)*, 32(2):5:1–5:29, 2014. <https://doi.org/10.1145/2619091>
- [13] OWASP. Mobile Application Security Verification Standard (MASVS) v2.0, 2024. <https://mas.owasp.org/MASVS/>

18 ■ Bibliography

- [14] OWASP. MASTG-KNOW-0018: WebViews in Android, 2024. <https://mas.owasp.org/MASTG/knowledge/android/MASVS-PLATFORM/MASTG-KNOW-0018/>
- [15] Flutter and Dart packages contributors. `webview_flutter`: A Flutter plugin providing a WebView widget on Android and iOS, 2025. https://pub.dev/packages/webview_flutter
- [16] Flutter and Dart packages contributors. `app_links`: Android App Links, Deep Links, iOS Universal Links and Custom URL handlers, 2025. https://pub.dev/packages/app_links
- [17] Flutter and Dart packages contributors. `url_launcher`: A Flutter plugin for launching a URL in the mobile platform, 2025. https://pub.dev/packages/url_launcher
- [18] PhishTank. PhishTank: Free community site for phishing data, 2025. <https://www.phishtank.com/>